PYTHON NOTES

Why python:

- Can run on different platforms
- Simple syntax
- Used in many modern applications

To check if python is installed?

➢ python --version

Syntax:

- Indentation is not for readability only.
- "#" used for comments
- No command for declaring variables.
- We can write many statements in one line using ";"

  Ex:
  print("hi there!");print("how are you?")

Output:

- Usage of end parameter:
  print('hi,whats up', end=" ")
  print("this text is in the same line)
  {we can use any character as a parameter for end}

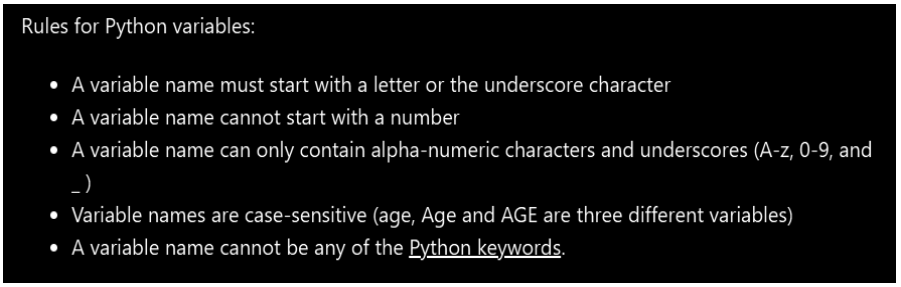
- Arithmetic calc. can be done in print statement:
  print(3+5)

Comments:

- Triple quotes can be used for multiline comments.
- Comments are used for better understanding of the code.

Variables:

- No command for declaring variable.
- Typecasting is possible.
- type() is used to identify type of variable.



- For multiple assigning:-
  x,y,z=”apple”,”banana”,”orange”
- One value to multiple variables is possible.
- Unpacking a list/tuple:
  Assigning elements to variable.
- Global variable:- can be used at any position…scope is global.
  Local variable:- can be used only inside the block….scope is limited.

Data Type:-

`x = "Hello World"`	str
`x = 20`	int
`x = 20.5`	float
`x = 1j`	complex
`x = ["apple", "banana", "cherry"]`	list
`x = ("apple", "banana", "cherry")`	tuple
`x = range(6)`	range
`x = {"name" : "John", "age" : 36}`	dict
`x = {"apple", "banana", "cherry"}`	set
`x = frozenset({"apple", "banana", "cherry"})`	frozenset
`x = True`	bool
`x = b"Hello"`	bytes
`x = bytearray(5)`	bytearray
`x = memoryview(bytes(5))`	memoryview
`x = None`	NoneType

- Random module use to get any random no. :-
  Import random
  print(random.randrange(1,10))

Strings:-

- Quotes can also be used inside the string but it should be different from the surrounding.
- 3 double quotes used for multiline string.
- “In” keyword used to check membership of any element or substring.
- Slicing used to get a substring from a given range.
  Syntax- str[start : stop : step]
- Some string functions:-
  - ➢ str.upper()
  - ➢ str.lower()
  - ➢ str.strip() #removes blankspaces from begining and of the string.
  - ➢ len(str)

- ➢ str.split() #split the string on the basis of parameter given in the function and gives output as a LIST.
- ➢ str.replace(x,y) #replaces substring x with substring y in the whole string.

- Concatination is used to join two strings by “+” symbol.
- Numbers cant be concatinated…so we use format() or fstring.
- F-String:-
  a=30
  txt=f”the price is {a*2} dollars”
  print(txt)
- Escape characters:-

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

- More string methods :-

https://www.w3schools.com/python/python_ref_string.asp

Boolean:-

- Returns True or False values.
- bool() is used to return true or false for any parameter.
- Generally returns true for every value of string or number or any other data type,but gives False for “0” or empty string.
- isinstance() gives a boolean value for the paramenter.

  Ex:- x=200
  print(isinstance(x,int))
  output=>True

Operators:-

- Types of operators:
  1. Arithmetic operators
  2. Assignment operators

3. Comparison operators
4. Logical operators
5. Identity operators
6. Membership operators
7. Bitwise operators

- Walrus operator (**:=**) is a type of assignment operator that assigns the value to variables as a part of a larger expression.

Ex - numbers = [1, 2, 3, 4, 27]

```
if (count := 9) > 8:
    print(f"List has {count} elements")
x=(a:=8)>5
print(x,a)
```

output:List has 9 elements

True 8

- Difference between is and ==

Is - Checks thief both variables points to the same object in memory.

== - Checks if the value of both variables are equal.

- Operator precedency is as follows:

Operator	Description
`()`	Parentheses
`**`	Exponentiation
`+x` `-x` `~x`	Unary plus, unary minus, and bitwise NOT
`*` `/` `//` `%`	Multiplication, division, floor division, and modulus
`+` `-`	Addition and subtraction
`<<` `>>`	Bitwise left and right shifts
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`==` `!=` `>` `>=` `<` `<=` `is` `is not` `in` `not in`	Comparisons, identity, and membership operators
`not`	Logical NOT
`and`	AND
`or`	OR

List:-

- List items are mutable,ordered and duplicable.
- Can store multiple types of data items.
- Indexing and slicing is used in list to access and traverse the element
- Range of items can also be updated using slicing. \
- Any item can be inserted at any index using insert().

  Syntax:- list_name.insert(index,item)

-